Installing at BL5A

This is the procedure for putting the stack in front of the real SPEC `kappa` session, the beamline's mar345 and the LakeShore 331. The simulation set-up is in the README; how to use the GUI is in usage.md.

Nothing here replaces SPEC: every motor move and count still goes through SPEC, so its kappa geometry, its `mar_ip7` guard and the operator's own SPEC terminal keep working. spec-ioc is a SPEC client next to that terminal.

1. What runs where

Process	Started by	Needs	Listens on
SPEC `kappa`, in server mode	the operator, on the SPEC host (its shortcut, section 5.3)	as today	its server port (`-S <port>`)
spec-ioc (`ioc/spec`)	systemd, `bl5a-spec-ioc`	TCP to SPEC's server port	Channel Access, prefix `BL5A:SPEC:`
mar345 IOC	the beamline, as today	the beamline's running IOC, `BL5A:MAR345:cam1:`	Channel Access
bl5a-qs	systemd, `bl5a-qs`	CA to both IOCs; **write access to the data root**	ZMQ 60715 (control), 60725 (documents)
bl5a-integrator	systemd, `bl5a-integrator`	ZMQ to qs 60725; **read and write access to the data root**	ZMQ 60735 (REP), 60745 (PUB)
GUI	the operator, the **BL5A** shortcut	CA to both IOCs and the ring PV; ZMQ to 60715, 60725, 60735, 60745; **read access to the data root**	—
MCP server (`gui/run_mcp.py`, optional)	Claude Code or Codex (section 4.8)	as the GUI	stdio

The three services run under the operator's user systemd on the host that runs bl5a-qs; `bl5a.target` starts and stops them together (section 5). The operator starts only SPEC, the GUI and the AI client.

All data goes under one data root: each queue item gets the folder `<root>/<series>/`, which holds its frames, its scans as `<series>.spec` and its integrated stacks (section 4.5). bl5a-qs makes the folder, mar345dtb writes the frames into it through the IOC's `FilePath`, the integrator writes the stacks and the GUI reads the frames, each by the same absolute path. Run them on the mar345 host, or mount the data directory at the same path on every host.

qs, the integrator and the GUI can share one host. Open the four ZMQ ports between hosts if they do not. The procedure below puts spec-ioc, qs, the integrator and the GUI on one host.

2. Get the source

```
git clone https://github.com/physwkim/bl5a_gui.git
```

The repository holds the SPEC driver and spec-ioc too (`drivers/spec`, `iocs/spec-ioc`). mar345-ioc and the crates the SPEC driver shares with the other epics-rs IOCs (`motor-common`, `ad-mar345`) come from https://github.com/epics-rs/epics-rs-iocs at the commit `Cargo.toml` pins (`910c8ca82`, on epics-rs 0.34.0); Cargo fetches them.

3. Build and install

Prerequisites on the host:

- Rust through rustup. This repository was built with 1.98.0; rsFAI needs 1.94 or later, and the epics-rs crates state no minimum.
- A C and C++ compiler: the `zmq` crate builds libzmq from source.
- Network access to github.com (rsFAI and epics-rs-iocs are git dependencies pinned to commits) and crates.io (bsrs 0.6.0, epics-rs 0.34.0). Without network on the beamline host, run `deploy/install.sh` once on a connected machine with the same `~/.cargo`, then run it on the host as `CARGO_NET_OFFLINE=true deploy/install.sh`.
- systemd with user services (`systemctl --user`); tested with systemd 255 (Ubuntu 24.04).

As the operator's user, not root:

```
cd <parent>/bl5a_gui
deploy/install.sh --python /path/to/python
```

`install.sh`:

- builds this directory's programs, `spec-ioc` among them, in release mode in `target/release/`, and `mar345-ioc` from epics-rs-iocs in `target/mar345-ioc/bin/`;
- copies the IOCs' database folders out of `~/.cargo` to `target/ioc-share/` (below);
- writes `~/.config/bl5a/bl5a.env` from `deploy/env/beamline/bl5a.env` (section 4);
- installs the user services `bl5a-spec-ioc`, `bl5a-mar345-ioc`, `bl5a-qs`, `bl5a-integrator`, `bl5a-sim` and the target `bl5a.target` in `~/.config/systemd/user/`, starting none of them;
- puts the **BL5A** shortcut, which starts the GUI, on the desktop and in the applications menu.

Run it again after an update: it rebuilds, rewrites the services and the shortcut, and keeps the settings files that exist. Then `systemctl --user restart bl5a.target`.

The IOC binaries find their database files through paths fixed at build time: `$(SPEC_IOC)` in this tree, and the crate folders under `~/.cargo` the rest name (`$(MOTOR)`, `$(SCALER)` and `$(ASYN)` for spec-ioc; `$(ADCORE)`, `$(ADMAR345)`, `$(ASYN)`, `$(CALC)` and `$(AUTOSAVE)` for mar345-ioc), which cargo deletes after a month unused. `install.sh` copies each of those folders to `target/ioc-share/<ioc>/` and lists them in `target/ioc-share/<ioc>.env`, which that IOC's service reads; a variable set in `spec-ioc.env` or `mar345-ioc.env` wins over it.

The GUI's Python

Tested with Python 3.12.14 and:

Package	Version
silx	3.1.2
PySide6	6.11.2
pyepics	3.5.7
pyzmq	27.2.0
msgpack	1.2.2
numpy	2.5.3
fabio	2026.6.0
qtpy	2.4.3 (optional)

`install.sh` warns when the Python given with `--python` (default `python3`) lacks one of them, and records it as `BL5A_PYTHON` in `bl5a.env`; the GUI's shortcut and the MCP server start that Python.

4. Configure

The services and the GUI's and MCP server's launchers read `~/.config/bl5a/bl5a.env`. systemd and `sh` both read it, so it holds only `KEY=value` lines: a value with spaces in double quotes, no `export`, comments on lines of their own. After a change: `systemctl --user restart bl5a.target`, and restart the GUI.

Key	Used by	At the beamline
`SPEC`	spec-ioc	`<spec host>:<port>` (section 4.1)
`BL5A_DATA`	bl5a-qs `--data-root`, bl5a-integrator `--root`	the data root (section 4.5)
`BL5A_PONI`	bl5a-integrator `--poni`	the PONI it starts with (section 4.6)
`BL5A_PYTHON`	the GUI and the MCP server	written by `install.sh`
`EPICS_CA_*`	all of them	the site's Channel Access settings, if any

`~/.config/bl5a/spec-ioc.env` and `mar345-ioc.env`, when they exist, are read after it by that IOC's service alone: the place for its own `EPICS_CAS_*` settings.

4.1 SPEC

Start the `kappa` session in server mode on a fixed port:

```
kappa -S2 <port>      # or -S <port>
```

Per SPEC's server manual (`help server` in SPEC): with `-S2` a limit hit stops all motors and returns SPEC to command level, as in a normal session; with `-S` (mode 1) only the motor at the limit stops. `-S` with no port takes the first free one in 6510–6530, which the IOC then has to be told; give a port. spec-ioc has not been run against either mode on the real SPEC; the simulation does not model the difference.

4.2 spec-ioc

The service `bl5a-spec-ioc` runs `spec-ioc st.cmd` in `ioc/spec` (`st.cmd` loads `db/bl5a.substitutions` by a relative path) and connects to `SPEC` from `bl5a.env`.

- It tries SPEC again every 2 s while SPEC is not there, so SPEC can start before or after it, and restart under it.
- It serves the motors `sy shold th tth nu scell martran marsdd marshut sx phi` as motor records `BL5A:SPEC:<motor>`, the counters `sec icxrd det ic1` as the scaler `BL5A:SPEC:scaler1` (S1–S4), the command records `BL5A:SPEC:Cmd*`, the scan command `BL5A:SPEC:ScanCmd` with SPEC's points of it in `BL5A:SPEC:Scan:Data` and `Scan:Cols` once it ends and in `Scan:Live:Data` and `Scan:Live:Cols` as SPEC takes them, and the console log `BL5A:SPEC:Log`.
- A motor record's stop sends SPEC `abort_all`, which stops every SPEC motor, as `^C` would.
- Use the site's Channel Access settings. The special ports in the README only keep the simulation off the network on a development box. If this host already runs an IOC on the default port, put another `EPICS_CAS_SERVER_PORT` in `~/.config/bl5a/spec-ioc.env` and add `host:port` to `EPICS_CA_ADDR_LIST` in `bl5a.env` and on the other clients.
- The IOC exits when its standard input ends, so the service runs it through `deploy/ioc-run`, which gives it the pipe `$XDG_RUNTIME_DIR/bl5a-spec-ioc/shell` as input. To type into the IOC shell: `echo dbl > $XDG_RUNTIME_DIR/bl5a-spec-ioc/shell`; the answer goes to `journalctl --user -u bl5a-spec-ioc`.

4.3 mar345

Use the IOC the beamline already runs (`BL5A:MAR345:cam1:`); do not start a second one on the same detector. bl5a-qs uses these PVs of it: `FilePath`, `FilePathExists_RBV`, `FileName`, `FileNumber`, `FullFileName_RBV`, `Acquire`, `Acquire_RBV`, `ArrayCounter_RBV`, `Erase`, `DetectorState_RBV`.

Before each frame bl5a-qs puts the item's folder in `FilePath` and fails the item with "mar345-ioc does not see the data folder ..." when `FilePathExists_RBV` then reads No: the IOC checks the path on its own host. This was run only against `mar345-ioc` from epics-rs-iocs (ADCore's rules in Rust); the C ADmar345 the beamline runs was not available to try.

The plans expect the settings `ioc/mar345/st.cmd` records as the beamline's (autosave and `mar345setup`). Check them once:

```
caget BL5A:MAR345:cam1:ScanSize BL5A:MAR345:cam1:ScanResolution \
      BL5A:MAR345:cam1:EraseMode BL5A:MAR345:cam1:AcquireTime \
      BL5A:MAR345:cam1:ImageMode BL5A:MAR345:cam1:NumImages \
      BL5A:MAR345:cam1:FilePath BL5A:MAR345:cam1:FileTemplate \
      BL5A:MAR345:cam1:AutoIncrement
```

Expected: 345 mm, 100 µm, no auto erase, AcquireTime 0 (the exposure is SPEC's count with the shutter open; Acquire only reads the plate out), single image, FileTemplate `%s%s_%4.4d` and AutoIncrement on. FilePath can read anything: bl5a-qs sets it per item.

Only with no IOC at all, run the one in this directory as the service `bl5a-mar345-ioc`:

```
cp deploy/env/beamline/mar345-ioc.env ~/.config/bl5a/   # then set MARSERVER and MAR_DATA in
it
systemctl --user enable bl5a-mar345-ioc.service       # bl5a.target now starts it too
```

`MARSERVER` is `<mar host>:5001`. `MAR_DATA` is an absolute path on the mar345 host, with a trailing `/`: the FilePath until bl5a-qs sets one per item. Its shell is `$XDG_RUNTIME_DIR/bl5a-mar345-ioc/shell`, as in section 4.2.

4.4 Temperature

Until the temperature IOC (`BL5A:TC:*`) exists, `insitu_step` runs the SPEC macros that `myinsitu` runs: it `qdo`s

```
/home/bl5a/specadm/spec6.06.11/macros/local_macros/temperature.mac
```

(the path in `myinsitu`; the constant `TEMPERATURE_MAC` in `qs/src/plans/mar.rs`) and calls `set_ramp`, `settemp` and `changec`. The file has to exist on the SPEC host. Its content is not in the archive this was built from, so the first in-situ run is also its first check against the real LakeShore (section 6, step 7).

4.5 bl5a-qs

The service `bl5a-qs` runs `bl5a-qs --data-root $BL5A_DATA` on the ports of section 1, with the log level `RUST_LOG=info,epics_ca_rs=warn`.

- At start-up it waits until every PV connects, naming the PV it waits for in its journal, so it comes up once both IOCs answer; until then the GUI shows qs not answering.
- It sets `EPICS_CA_PUT_TIMEOUT=1e6` unless set: a monitor-count exposure is one put that completes minutes later.
- `BL5A_DATA` (`--data-root`) is an existing absolute directory without spaces, the root of the data of every item that does not name its own. An item's `md.data_root` names its own; the GUI puts the folder of its Data box there (empty: none). Under the root each item gets one folder, made when the item starts:

  ```
  <root>/<series>/
      <series>.spec             the item's scans
      <name>_0001.mar3450 …     its frames
      <series>.h5               its stack, then <series>~2.h5, … (section 4.6)
  ```

  In the stack's name a byte other than a letter, digit, `_`, `-` or a `.` after the first is written `%XX`: series `LiFePO4(1C)` gives `LiFePO4%281C%29.h5`.

  The folder is the item's series (`md.series`), or the plan's name when it has none, so the items of one series share it. A series is one folder name: not empty, `.` or `..`, and without `/`. The folder `<root>/<series>` holds no space or control character and is at most 200 bytes: mar345dtb takes a frame's path as one word of its `COMMAND SCAN <path>` line, which ADmar345 sends from a 256-byte buffer, and a space would split it. The GUI refuses any other before queueing; bl5a-qs fails such an item when it runs, before it moves anything, and refuses such a `--data-root` at start. An item whose root does not exist fails at its start with `device state error: data folder: <root>/<series>: the data root <root> does not exist`.
- `<series>.spec` is the SPEC data file of the folder's scans. SPEC runs as a server and the alignment scans reach no file of SPEC's, so bl5a-qs appends every scan to it, the step scans it runs and the fly scans SPEC runs: one `#S` block each, readable by splot and PyMCA. The

file is created if missing; an existing one gets a new `#E` header and its numbering continues from its last `#S`. Keep SPEC's own `newfile` out of the data root.

- It differs from SPEC's own file: the labels are the mnemonics (`th`, `det`); a fly scan has no `Epoch` column, since its points are timed when they reach bl5a-qs, not when SPEC took them; the columns between `Epoch` and the detector are in name order, not counter order; there are no `#O`, `#P`, `#G` or `#Q` lines. The exposures' `#MAR345` lines and `loopscan` go only to SPEC's own file, when one is open.

4.6 bl5a-integrator

The service `bl5a-integrator` runs `bl5a-integrator --poni $BL5A_PONI --root $BL5A_DATA`, with npt 2000 and bl5a-qs on the same host (`tcp://localhost:60725`).

- `--root` is bl5a-qs's `--data-root`, an absolute path. The integrator files each frame's row in the stack of its series in the item's data folder, which the run's start document names, so items with a root of their own get their stacks there too. At start it lists the stacks already in the folders under `--root`; another root's stacks are listed once a row goes to them. It is the stacks' only writer.

- A stack holds one series on one q axis: `<series>.h5`. While the newest stack of the series in the folder bins on the current axis, rows go to it. When npt, or a PONI that moves the plate's far rim, changes the axis, the next row starts `<series>~2.h5`, then `~3`, and so on, and the old stacks keep their axis. Going back to an earlier axis also starts a new one. Each row records the PONI it was integrated with.

- `BL5A_PONI` and npt are what it starts with. The GUI's Integration box sends another PONI and npt (the REP `set`); they take effect after the frames already waiting to be integrated, and are lost when the integrator restarts.

- The PONI is for the full 345 mm plate. **The integrator applies one PONI to every frame**, while the plans take frames at four mar345 positions (`mar_ip7` arguments: martran, marsdd, …):

Used by	mar_ip7
Battery, standard	`-85 435 100 330 50 1.2 -6.8`
Battery, high_q_asymm	`-85 435 110 330 80 1.2 -6.8`
Battery, low_q_symmetric	`160 435 36 345 54 1.2 -6.8`
In-situ (`mymar_in`)	`25 440 42.4 340 81.8 1.2 -6.8`

  Calibrate a PONI for the position in use and apply it in the GUI's Integration box when the experiment changes position, before the first frame there. Frames taken at another position get wrong q values; the image tab's rings come from the same PONI.

- Check the PONI and the decoding on a real BL5A frame before the first run: the overflow handling was tested only on a small frame written by fabio (`integrator/tests/fixtures/make_mar.py`).

4.7 GUI

Edit `gui/bl5a_gui/config.py`:

Setting	At the beamline
`RING_PV`	`BL5A:userCalcOut1.VAL` (SPEC's `ringI`); keep
`SHUTTER_PV`	`None`: the mar345 shutter is PFCU1 blade 3 on SPEC's serial line and has no PV
`TEMP_SP_PV`, `TEMP_RBV_PV`, `TEMP_RAMP_PV`	`None` until the temperature IOC exists
`PONI`	the image tab's PONI while the integrator does not answer; once it answers, the integrator's is used
`QS_*`, `INTEGRATOR_*`	or pass `--qs-control`, `--qs-documents`, `--integrator-rep`, `--integrator-pub`
`USER`	or pass `--user`; recorded with each queued item

The Data box's folder and the Integration box's PONI file are kept per user in Qt's settings (`~/.config/bl5a/bl5a-gui.conf` on Linux).

The **BL5A** shortcut runs `deploy/bl5a-gui`, which reads `bl5a.env` and starts `gui/run_gui.py` with `BL5A_PYTHON`; options after it go to `run_gui.py` (`deploy/bl5a-gui --user kim` in a terminal). Set lasting choices in `config.py`: `install.sh` rewrites the shortcut.

4.8 MCP server

`gui/run_mcp.py` serves the GUI's controls to Claude Code or Codex over stdio (usage.md). It needs the GUI's Python plus two packages, tested at these versions:

```
python -m pip install mcp==2.3.0 pillow==12.3.0
```

The client starts it through `deploy/bl5a-mcp`, which, like the GUI's launcher, reads `bl5a.env`, so the client needs no environment of its own. It takes the GUI's options (`--qs-control`, `--qs-documents`, `--integrator-rep`, `--integrator-pub`, `--user`) and reads the same `config.py`. Pass `--user claude` (or `codex`) so the queue's User column tells the AI's items from the operator's. It reads the Data folder from the operator's Qt settings, so run it as the operator's user.

Claude Code, from the directory it will be started in (`--scope project` writes `.mcp.json` there):

```
claude mcp add --scope project bl5a -- <parent>/bl5a_gui/deploy/bl5a-mcp --user claude
```

Codex, in `~/.codex/config.toml`. A motor move can take up to 300 s and `spec_command` waits up to 600 s, so raise the tool timeout:

```
[mcp_servers.bl5a]
command = "<parent>/bl5a_gui/deploy/bl5a-mcp"
args = ["--user", "codex"]
tool_timeout_sec = 660
```

Check it with `/mcp` in either client: `bl5a` connected, 35 tools.

5. Starting and stopping

5.1 The services

Once the first run (section 6) has passed:

```
systemctl --user enable --now bl5a.target
```

starts spec-ioc, bl5a-qs and bl5a-integrator now and at every login of the operator's user, and `bl5a-mar345-ioc` with them where it is enabled (section 4.3). systemd starts the IOCs, then bl5a-qs, then the integrator, and starts one that fails again after 5 s. To start them at boot without a login, and keep them running after logout:

```
sudo loginctl enable-linger <user>     # loginctl show-user <user> -p Linger: yes
```

Then:

```
systemctl --user status 'bl5a*'              # which run
journalctl --user -u bl5a-qs -f              # one service's log, as it comes
systemctl --user restart bl5a.target         # after editing ~/.config/bl5a/
systemctl --user restart bl5a-spec-ioc       # one service
systemctl --user stop bl5a.target            # stop them until the next start or login
systemctl --user disable --now bl5a.target   # stop them, and not at the next login
```

5.2 What the operator starts

1. SPEC, on the SPEC host: `kappa -S2 <port>` in a terminal, or its shortcut (section 5.3). spec-ioc connects to it within 2 s, whenever it starts.
2. The GUI: the **BL5A** shortcut.
3. Claude Code or Codex, when used, in `<parent>/bl5a_gui`; it starts the MCP server itself.

The queue and the integrator keep running when the GUI is closed; a GUI started later picks up the queue, the running plan and the stacks. If spec-ioc restarts, the SPEC console tab marks it with `--- spec-ioc restarted ---`.

5.3 Desktop shortcuts

A shortcut is a desktop entry, a `.desktop` file: in `~/.local/share/applications/` it shows in the applications menu, in the desktop folder (`xdg-user-dir DESKTOP`, as a rule `~/Desktop`) as an icon. `install.sh` writes the **BL5A** one, `bl5a-gui.desktop`, to both:

```
[Desktop Entry]
Type=Application
Name=BL5A
Comment=Queue BL5A alignments and mar345 measurements
Exec=<parent>/bl5a_gui/deploy/bl5a-gui
Icon=applications-science
Terminal=false
Categories=Science;Physics;
```

SPEC needs a terminal of its own, so its shortcut, on the SPEC host, has `Terminal=true`, and the window it opens is SPEC's terminal. As `kappa.desktop`:

```
[Desktop Entry]
Type=Application
Name=SPEC kappa (server)
Exec=/path/to/kappa -S2 <port>
Icon=utilities-terminal
Terminal=true
```

Claude Code in this directory, as `claude-bl5a.desktop`; `Path=` is the directory it starts in, where it finds `.mcp.json`, and `command -v claude` gives the `Exec=` path (Codex alike):

```
[Desktop Entry]
Type=Application
Name=Claude Code (BL5A)
Path=<parent>/bl5a_gui
Exec=/home/<user>/.local/bin/claude
Icon=utilities-terminal
Terminal=true
```

To install one of them:

```
desktop-file-validate kappa.desktop             # no output: well formed
cp kappa.desktop ~/.local/share/applications/   # the applications menu
d=$(xdg-user-dir DESKTOP)
cp kappa.desktop "$d/" && chmod +x "$d/kappa.desktop"
gio set "$d/kappa.desktop" metadata::trusted true                     # GNOME
gio set "$d/kappa.desktop" metadata::xfce-exe-checksum \
    "$(sha256sum "$d/kappa.desktop" | cut -d' ' -f1)"                 # Xfce
```

A desktop runs an icon without asking only once it is executable and trusted. GNOME trusts it by the flag (right-click → **Allow Launching** sets it too), Xfce by the file's SHA-256, so after editing the file on the desktop, run its Xfce line again. Where the desktop shows no icons, the applications menu entry is enough: search for its name.

6. First run, step by step

Do these in order and go on only when a step matches SPEC.

1. **Read only.** Start SPEC in server mode, then spec-ioc alone (`systemctl --user start bl5a-spec-ioc`), then the GUI from the **BL5A** shortcut. Without bl5a-qs answering, the GUI's motor buttons and console are disabled, so nothing can be sent from it. Compare the motor panel, and `caget BL5A:SPEC:<motor>.RBV`, with `wa` in the SPEC terminal. Type a command in the SPEC terminal and see it in the GUI's SPEC tab.
2. **Commands.** spec-ioc sends each command with `SPEC_IOC_CMD_DONE = 0` before it and `SPEC_IOC_CMD_DONE = 1` after it, and calls the command failed when the marker is not 1 afterwards. This catches a macro that stops at `exit`, such as `mar_ip7`'s two-theta check. Check that behaviour once on the real SPEC (`exit` only returns SPEC to its prompt). After `caput -c -w 10 -S 'BL5A:SPEC:Cmd.VAL$' 'exit'`, `caget BL5A:SPEC:Cmd:Status` must read

Failed. After `caput -c -w 10 -S 'BL5A:SPEC:Cmd.VAL$' '2+2 # a comment'`, it must read OK, and `caget -S BL5A:SPEC:Cmd:Reply` must give 4. If `exit` reads OK, SPEC ran the marker line after the stop, so plans could go on past a refused `mar_ip7`: do not queue plans. A Reply other than 4 means SPEC replies with the marker's value. The plans do not use reply values, but the console then shows that value instead.

3. **A small move.** Start bl5a-qs (`systemctl --user start bl5a-qs`) with an empty queue. The GUI's motor buttons enable once SPEC is idle. Set a small step on a safe motor, press ◀ then ▶, and check the position in SPEC (`wm <motor>`).
4. **A count.** `caput -c -w 10 BL5A:SPEC:scaler1.TP 1` and `caput -c -w 10 BL5A:SPEC:scaler1.CNT 1`, then `caget BL5A:SPEC:scaler1.S1 BL5A:SPEC:scaler1.S2 BL5A:SPEC:scaler1.S3 BL5A:SPEC:scaler1.S4` (sec, icxrd, det, ic1). Compare with `ct 1` in SPEC.
5. **A fly scan.** spec-ioc reads a scan's points from SPEC's `NPTS`, `SCAN_COLS` and `SCAN_D`, which only the simulation has stood in for so far. On a safe motor: `caput -c -w 60 -S 'BL5A:SPEC:ScanCmd.VAL$' 'clup th -0.1 0.1 2 0.1'`. `caget BL5A:SPEC:Cmd:Status` must read OK; if it reads Failed, `caget -S BL5A:SPEC:Cmd:Reply` says what did not match. `caget -S BL5A:SPEC:Scan:Cols` must list the columns of the scan header SPEC printed, and `caget BL5A:SPEC:Scan:Data` must hold `NPTS` (`p NPTS` in SPEC) times that many values, its first row being `p SCAN_D[0]`.
6. **An alignment.** Queue Coin cell or Thin film for the sample on the stage. Compare each stage's CEN and FWHM in the GUI with SPEC's own scan over the same range (the scan title in the Scan tab gives it) and SPEC's `CEN`; a stage whose scan had a point replaced by the outlier filter can differ (usage.md). Compare an EDGE stage with where SPEC's scan falls through half its maximum, not with SPEC's `CEN`.
7. **A frame.** Start the integrator (`systemctl --user start bl5a-integrator`). Battery, Single shot, a short Texpo and a test name. Check that the GUI's Data box shows the item's folder for mar345 without "missing", that the file appears in that folder, that the Image tab shows it, and that the Stack tab gets the row and lists the stack in the same folder.
8. **Temperature.** One in-situ row with a small T change and wait on, watching the LakeShore's front panel; this runs `temperature.mac`. Moving the LakeShore from SPEC's ser 9 to the temperature IOC (`BL5A:TC:*`) comes later: first change the ser 9 macros to use `BL5A:TC:*`, then hand the port to the IOC.

When every step matches, `systemctl --user enable --now bl5a.target` (section 5.1).

7. When something is wrong

Symptom	Where to look
A service is failed or keeps restarting	`systemctl --user status 'bl5a*'`, then `journalctl --user -u <service>`
bl5a-qs is active but the GUI shows qs not answering	its journal's last `waiting for` line names the PV that does not connect: IOC down or `EPICS_CA_ADDR_LIST` in `bl5a.env`
The services stop at logout, or do not start at boot	`sudo loginctl enable-linger <user>` (section 5.1)
The **BL5A** shortcut opens nothing	run `deploy/bl5a-gui` in a terminal for its error; `BL5A_PYTHON` in `bl5a.env`
The desktop icon opens as text, or asks before running	`chmod +x` and the `gio set` lines of section 5.3; on Xfce, again after each edit of the file
GUI controls stay grey	the status line: QS not answering, SPEC busy or disconnected, or a plan running

Symptom	Where to look
"mar345 Acquire: no frame scanned" or "… the detector did not take it"	the mar345 IOC and mar345dtb
Image tab empty, its info line shows a file error	the frame path is not readable on the GUI host
Stack tab gets no row, Messages show `integrator:`	the integrator's log; the frame path on its host
An item fails with "data folder: … the data root … does not exist"	the data root on bl5a-qs's host, or the Data box's folder
An item fails with "data folder "…": must be at most 200 bytes without spaces …"	the series or the data root holds a space, or the path is too long
An item fails with "mar345-ioc does not see the data folder …"	the data root is not mounted at the same path on the mar345 IOC's host
A plan fails on `mar_ip7`	tth: UPWARD geometry needs tth ≥ 80